

MICROPROCESOARE SI LIMBAJE DE ASAMBLARE

Calculatoare 3 2025-2026

Curs 6 Sapt 6 04 noiembrie 2025 16:00-18:00

serban@upit.ro

Modul de notare:

Examen:	50%	
Laborator:	30%	A2
Test (Midterm):	20%	A1

Planificarea activitatilor:

Laborator:

- saptamanal, conform orarului;
- sapt. 13, 14 refaceri laboratoare;

Curs:

- saptamanal, conform orarului;

Test (Midterm): in saptamana a 11-a (**16 decembrie 2025**)

OBS. Este nevoie de nota minima 5 la toate activitatile (semestru – A1, A2 si sesiune – Examen!!! pentru promovare)

Continut

Introducere

Evolutia microprocesoarelor

Criterii de clasificare ale microprocesoarelor

Structura microprocesoarelor

Mecanisme specifice Microprocesoarelor

Microprocesoare pe 8 biti

Familia de microprocesoare x86

Familia de procesoare ARM

Bibliografie

The Design of a Microprocessor – Wilhelm G. Spruth – Springer

Microprocessor Design – Grant McFarland – McGraw-Hill – 2006

Microprocessor Interface Design – J.D. Nicoud – Chapman & Hall

Microprocessors and Microcomputers Hardware and Software - Ronald J. Tocci – Pearson

Intel Microprocessors – Barry Brey

Z-80 Microprocessor Architecture, Interfacing, Programming and Design - Ramesh Gaonkar

Introduction to 80X86 Assembly Language and Computer Architecture - Richard C. Detmer - Jones & Bartlett Pub (2001)

X86 Assembly Language and C Fundamentals – Joseph Cavanagh – CRC Press

ARM Assembly Language Fundamentals and Techniques, Second Edition - William Hohl

Fundamentals of Embedded Software Where C and Assembly Meet – Daniel Wesley Lewis

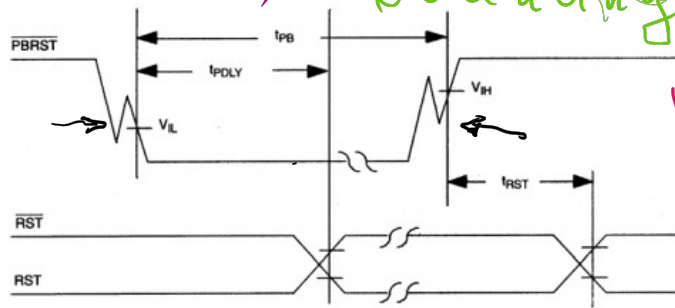
Mecanismul de RESET

- Reset manual
- Power On Reset POR
- Supervisor
- Brown-out
- Watchdog

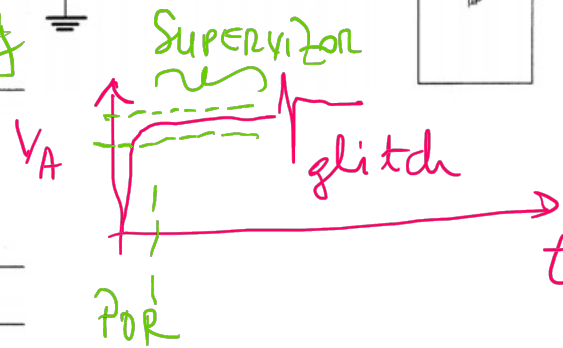
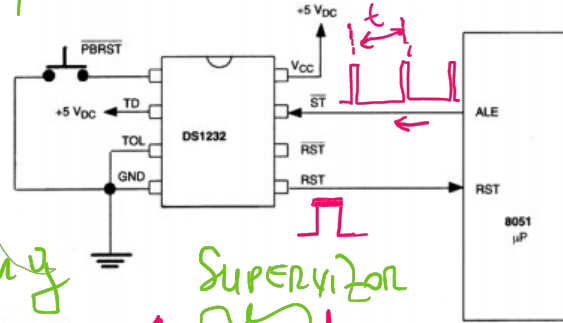
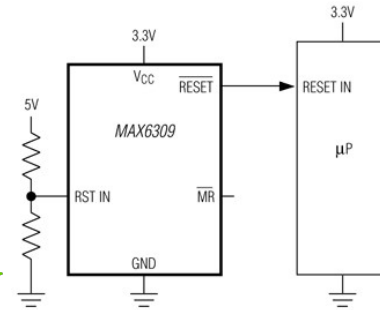
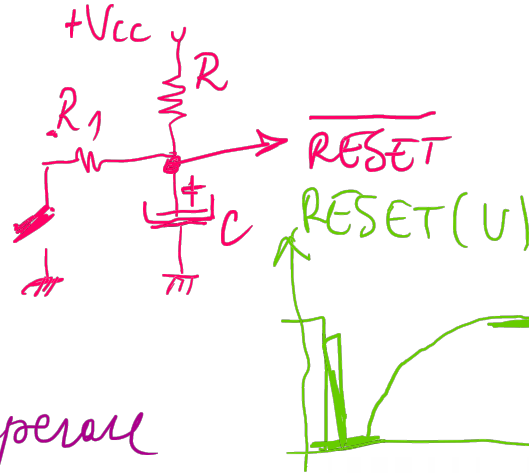
Condiție inițială de operare
(prin reset)

$PC \leftarrow \text{adr. start (reset)}$

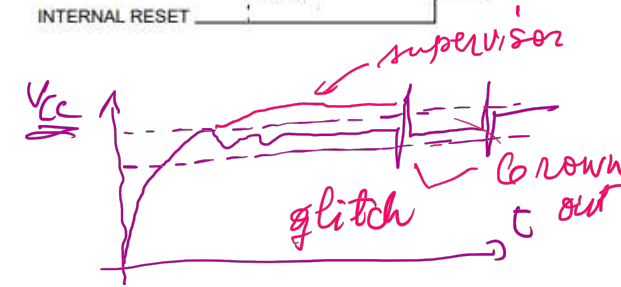
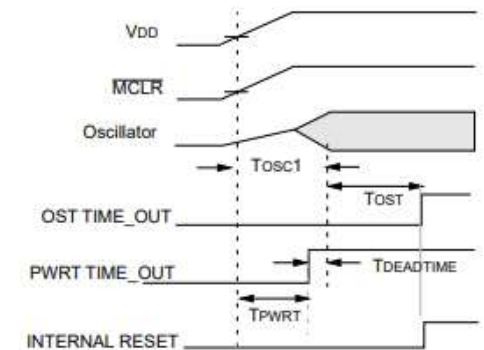
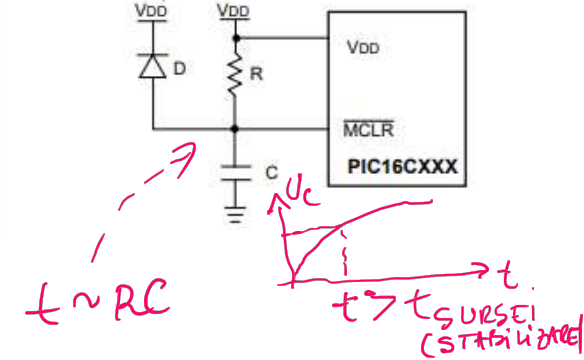
(μP z-80 $\rightarrow$ 0000h)
(μP x86 $\rightarrow$ FFFFFFFFh)
32bit

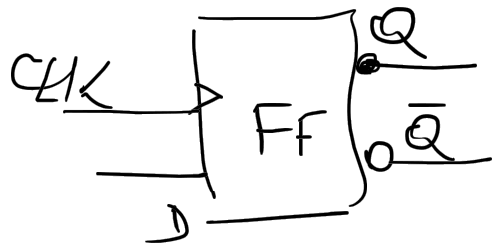


Mecanisme specifice in procesoare

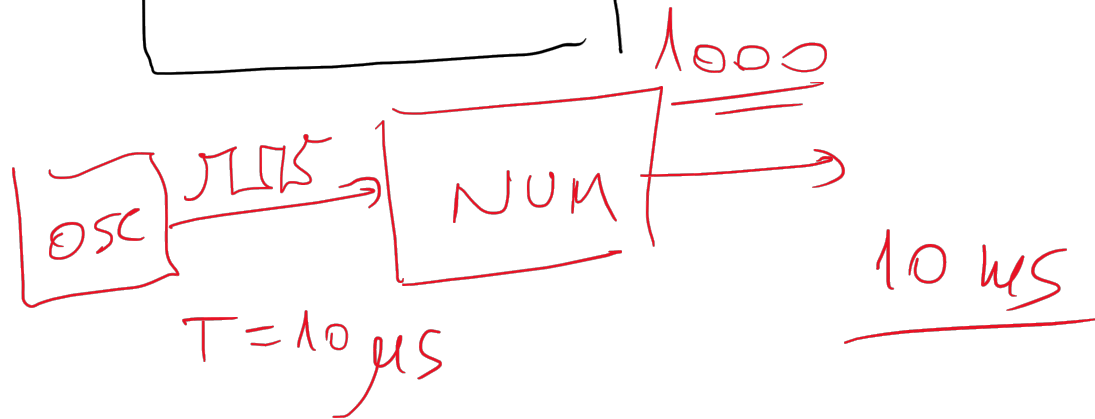
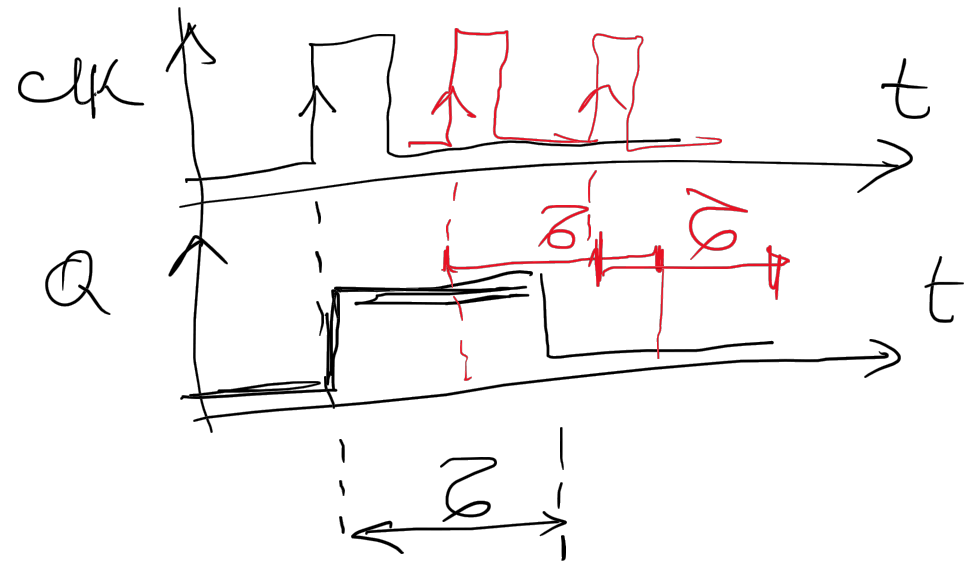
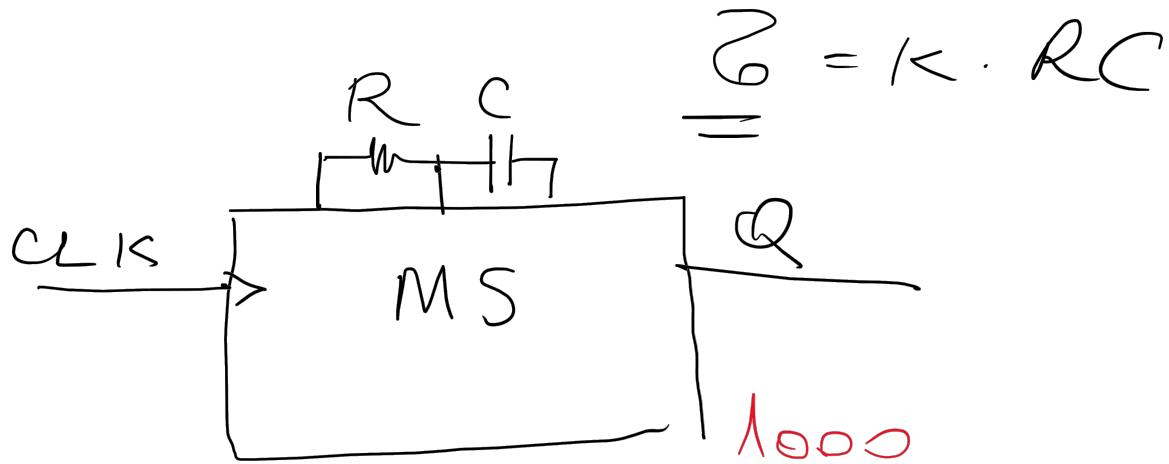


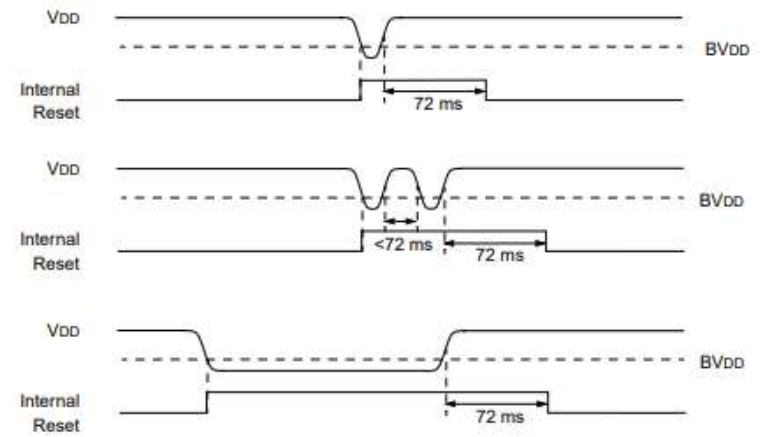
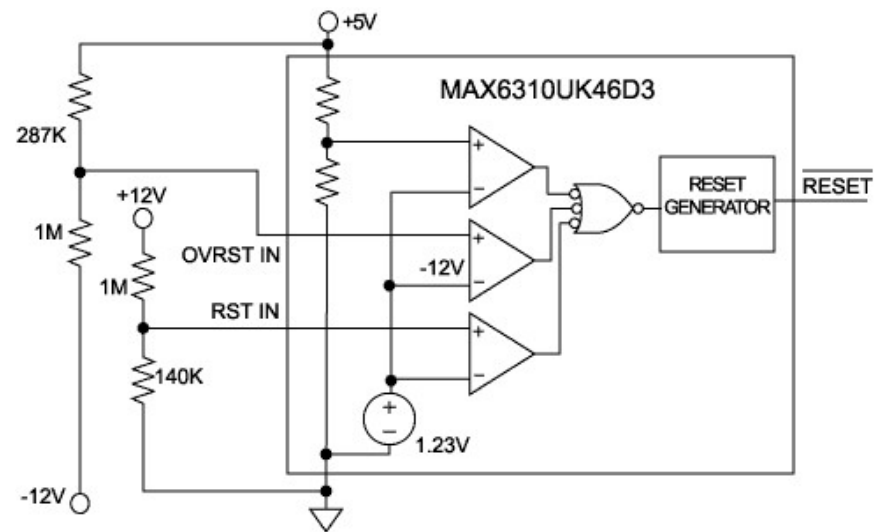
$$U_{nom} \in [0,5 \dots 0,9] U_{abs} \quad U_{nom} \approx 5V \quad 10\mu F / 25V \quad 10\mu F / 10V \leftarrow U_{abs}$$

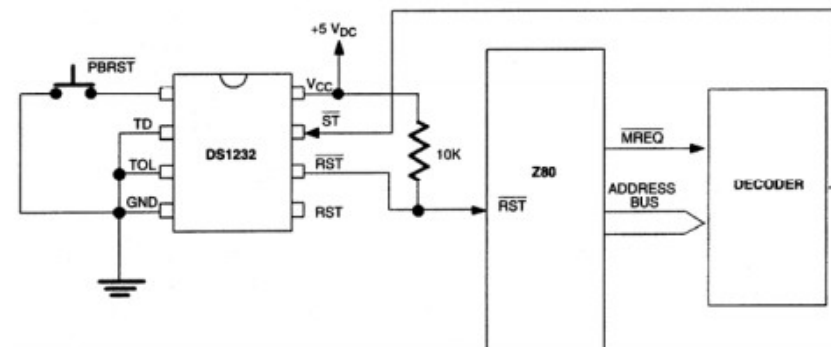
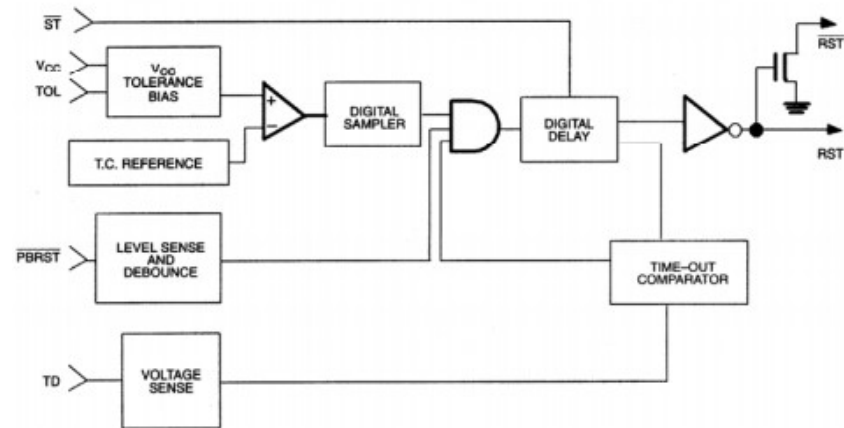
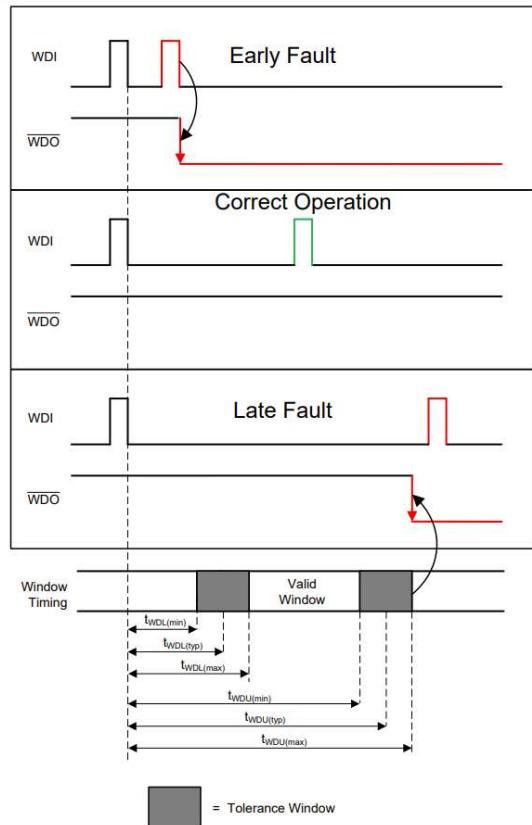




flip-flop







Mecanismul de operare cu stiva si registrul SP

Stiva este o zonă de memorie de tip RAM, de obicei externă procesorului, folosită pentru:

- situația apelării subrutinelor; o subrutina este formata dintr-un grup de instructiuni a caror rulare se repeta intr-un program; pentru a nu ocupa memorie, grupul respectiv de instructiuni se individualizeaza intr-o zona de memorie, marcandu-se adresa de inceput printr-o eticheta, iar sfarsitul logic printr-o instructiune specializata de revenire (RETURN); subrutina va fi apelata din program de fiecare data cand este necesar printr-o instructiune de tip CALL eticheta (adresa subrutinei).
- tratarea întreruperilor și excepțiilor programului de lucru. Întreruperile respectiv excepțiile sunt evenimente deosebite care apar în timpul execuției. O întrerupere este o semnalizare făcută procesorului de dispozitive externe acestuia (circuite de I/O). Spre exemplu, scurgerea unei perioade de timp poate fi semnalizată printr-o cerere de întrerupere. Excepțiile sunt semnalizări adresate procesorului de anumite componente interne, de exemplu forțarea operației de împărțire prin 0. Pentru toate situațiile specificate proiectantul software-ului trebuie să scrie subrutine pentru tratarea situațiilor deosebite apărute. Trecerea la tratarea situațiilor menționate se face de către procesor prin apeluri speciale ale subrutinelor scrise în acest scop.

OBS. Situațiile mentionate mai sus folosesc stiva în mod automat.

- Salvarea / memorarea de informații la cererea programatorului sau automat (continuturi de registri, de locații de memorie) prin executia de instructiuni specifice (PUSH la depunerea în stiva, POP la extragerea din stiva).
- Schimburi de continuturi între registri.

Stiva si registrul SP

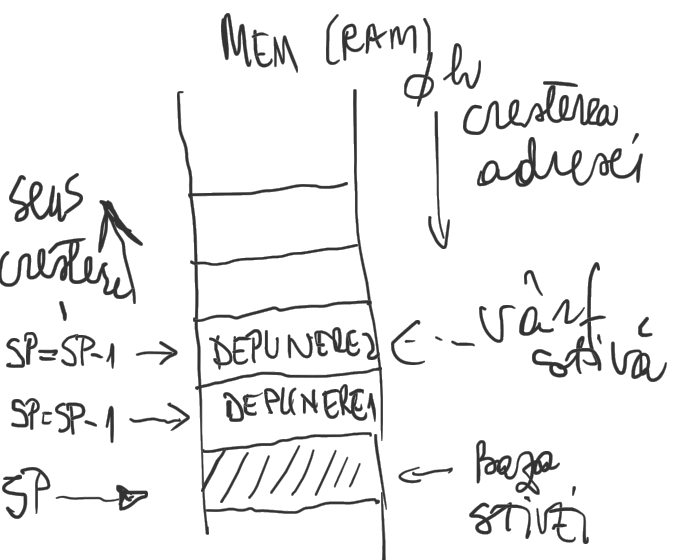
Stiva este adresată printr-o tehnică LIFO (last in – first out) prin utilizarea registrului SP (stack pointer – indicatorul varfului stivei). În practică se întâlnesc două modalități de creștere a stivei (la depunerea în ea):

- ✓ Către adrese mici de memorie/înapoi; În această situație la fiecare depunere în stivă, SP se decrementează (la extragere SP se incrementează);
- ✓ Către adrese mari; În acest caz la depunere SP se incrementează, iar la extragere SP se va decrementa.

SP-ul arată întotdeauna varful stivei, adica adresa ultimei depuneri (sau a bazei stivei). Există și posibilitatea fixării bazei stivei. Aceasta se face prin încărcarea SP-ului cu adresa absolută de la care urmează să crească stiva.

Stiva poate fi internă sau externă procesorului cu care operează, astfel cele mai multe procesoare pot forma stivă în memoria externă adresabilă. Dat fiind faptul că stiva trebuie scrisă, respectiv citită, zona de memorie în care acesta este implementată este de tip RAM. Pentru această situație (stivă externă) dimensiunea stivei depinde de limitele memoriei și de spațiile alocate în aceasta pentru diversele aplicații.

Există și procesoare la care stiva se formează în zone de memorie special alocate și aflate pe același cip. Dimensiunea acestor zone de memorie specifice este limitată, doar câteva locații disponibile (de ordinul zecilor). Procesoarele aflate în această situație sunt astfel limitate în posibilitățile lor numărul de depuneri ce se pot face în stivă fiind mărginit de numărul de locații al acesteia.



SP - se decrementează
la depunerea în stivă

SP - se incrementează
la extragerea din stivă

Utilizatorul “simte” acest lucru prin limitarea numărului de subrutine imbricate (apelate unele din altele) pe care le poate folosi. În această categorie intră procesoarele realizate de firma MICROCHIP, denumite PIC.

Nu există procesoare care să dispună de ambele modalități de formare a stivei, adică atât în memoria externă cât și în cele interne dedicate. Pentru ambele categorii de procesoare relativ la modul de formare a stivei accesarea prin utilizarea registrului SP se poate face fie prin decrementarea SP-ului, fie prin incrementarea acestuia (la depunerea în stivă). Pentru un anumit procesor este bine definit acest mecanism de operare cu stiva utilizând registrul SP.

De obicei, salvarea în stivă a conținuturilor registrilor se face la începutul derulării subrutinelor, scopul fiind acela de a memora informațiile din ei, în ideea eliberării acestora pentru executia subrutinei. Salvarea se face manual prin instrucțiuni PUSH ale utilizatorului. La sfârșitul subrutinei, se restaurează din stiva conținuturile anterioare ale registrilor, în ordine inversă depunerii, folosind instrucțiuni de tip POP. Se respectă tehnica de adresare LIFO a stivei.

Exemplu de salvare si restaurare in si din stiva folosind instructiunile PUSH si POP

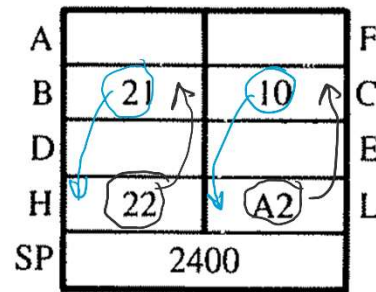
*

PROGRAM

2000 LD SP, 2400H (Baza stivei) LD SP, 2400H
 2003 LD HL, 22A2H
 2006 LD BC, 2110H
 2009 LD A, (HL)
 200A OR A
 200B ----->

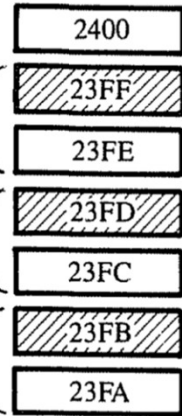
2010 PUSH BC
 2011 PUSH HL
 2012 PUSH AF

2035 POP AF
 2036 POP HL
 2037 POP BC



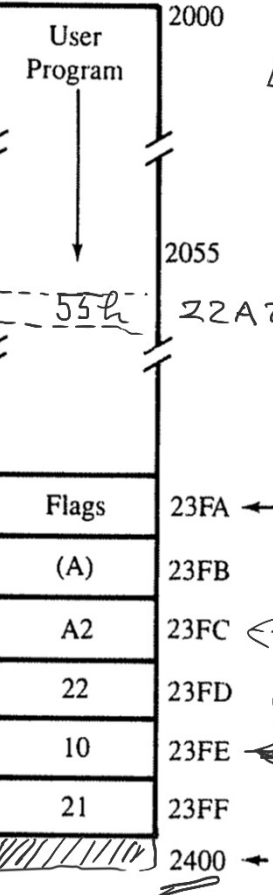
LITTLE ENDIAN

Stack Pointer Contents



OR A - $A \leftarrow A \vee A$

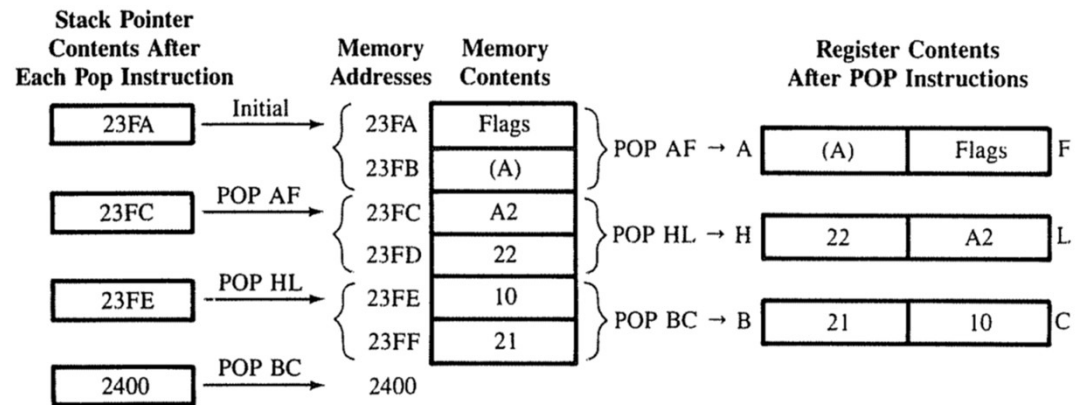
Memory Contents



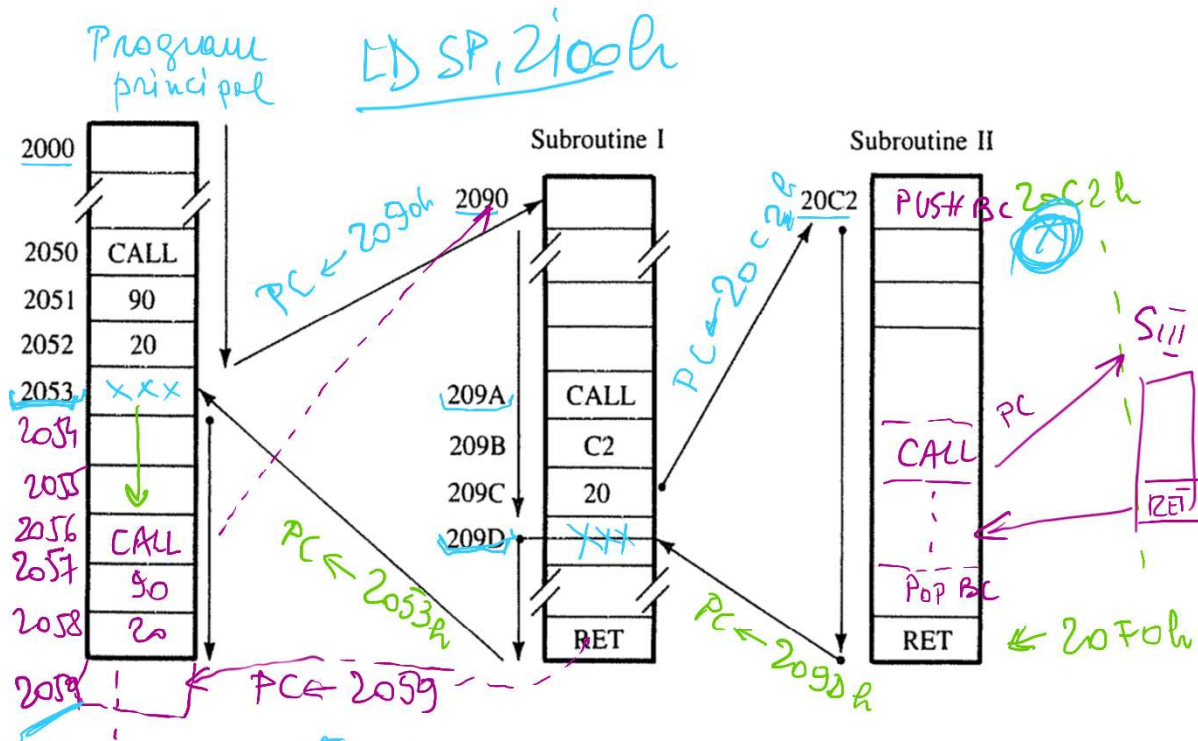
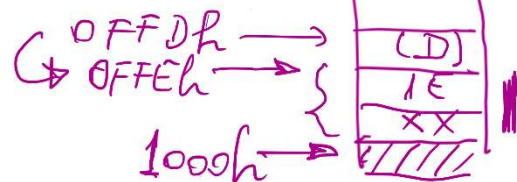
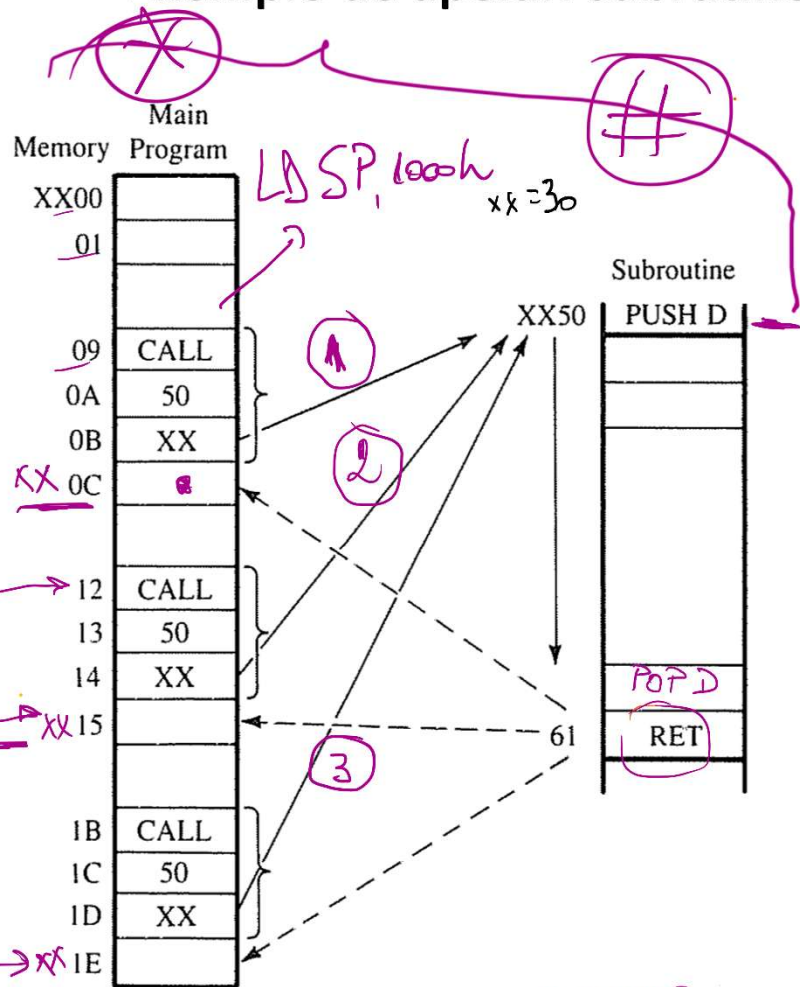
LD A, (HL)
 $A \leftarrow 55h$

LIFO

28 Oct 2025

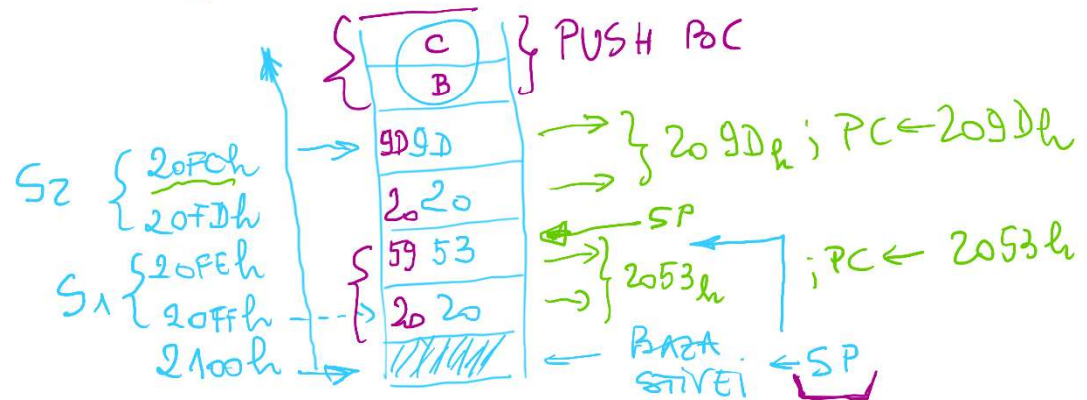


Exemple de apeluri subrutine



Stack diagram showing memory addresses and values:

209Dh	90 9D
20FDh	20 20
20FEh	59 53
20FFh	20 20
2100h	



Registrul indicatorilor de conditii (F – Flags sau PSW – Program Status Word)

C – carry	;indica depasirea valorii 255, atunci cand se lucreaza pe 8 biti – fara semn;
O – overflow	;indica depasirea intervalului [-128; 127] cand se lucreaza in CC2 pe 8 biti – cu semn;
S – sign	;S = 1 pentru numere negative si S = 0 pentru numere pozitive si 0 (S este bcms din rezultat);
P – parity	;P=1 indica un numar par (even) de biti '1 – logic' in rezultat; P = 0 la rezultat impar (odd);
H – half-carry	;H = 1 indica un transport intre cei 2 nibble ai rezultatului;
N – add/subtract	;N = 0 la adunare si N = 1 la scadere (utilizat la operatii cu reprezentari BCD pentru operanzi)
Z – zero	;Z = 1 pentru rezultate nule, Z=0 la rezultate nenule

$$\begin{array}{r} \rightarrow 1011.0101 = B5h = \\ \downarrow \\ 10010101 \quad (fara\ semn) \\ \hline 0100.1011 = 4Bh = \\ \hline \end{array}$$

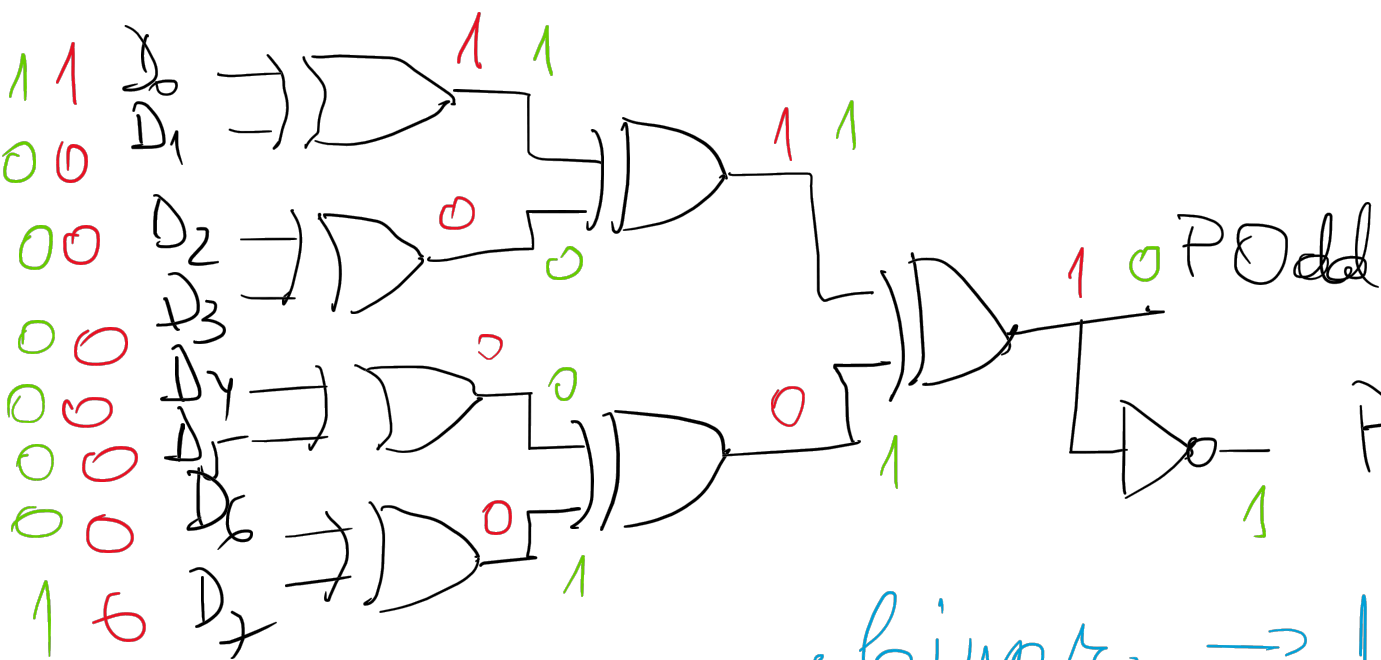
$$181_{(10)} - 148_{(10)} = 133_{(10)}$$

OBS. 1

Procesorul nu cunoaste modul de reprezentare al operanzilor pe care utilizatorul/compilerul vrea sa ii proceseze (cu sau fara semn)! Modul de reprezentare cu semn (adica in CC2) sau fara semn este doar o conventie a programatorului/compilerului. Procesorul insa ofera un rezultat in urma procesarii indiferent de modul de reprezentare si indica in F amanunte despre rezultat. Este sarcina utilizatorului/compilerului sa faca interpretarea rezultatului procesarii operanzilor si a valorilor indicatorilor de conditii!

OBS. 2

Indicatorii de conditii sunt afectati de executarea operatiilor aritmetico-logice, vechile valori ale acestora pierzandu-se. Este sarcina programatorului/compilerului sa verifice starea indicatorilor de conditii dupa fiecare operatie aritmetico-logica. Indicatorii de conditii raman neafectati, adica memoreaza starea lor anterioara, atunci cand sunt executate alte tipuri de instructiuni (altele decat aritmetico-logice). Starea logica a indicatorilor de conditii se poate testa de catre programator prin instructiuni. Nu toti indicatorii de conditii se pot testa direct prin instructiuni. Indicatorii de conditii care nu se pot testa direct, se pot testa indirect prin interogarea registrului F. Indicatorul C se poate modifica prin instructiuni (se poate seta adica C = 1 sau reseta C = 0). C nu este afectat de instructiunile INC si DEC!!!



0001 0100 (14)
BCD

0001 1000 (18)
BCD

0010 1100 +
0110 @ 4th BCD

0011 (3) 0010 (2)
0010.0011 (2)

35 (10)

0001 0001

Binary $\rightarrow 17_{(10)}$

BCD $\rightarrow 11_{(10)}$

Binary $\rightarrow 18_{(10)}$

BCD $\rightarrow 12_{(10)}$

0001 0010

0010.0011

23 (10)

Registrul PC – Program Counter – Registrul numarator de program

Rolul acestui registru într-un procesor este acela de a indica adresa de memorie de la care procesorul urmează să preia codul binar al următoarei instrucțiuni de executat. Acest registru există în orice procesor (eventual cu alta denumire – de exemplu – IP instruction pointer). Dimensiunea în biți a acestui registru este de obicei egală cu dimensiunea magistralei de adrese a procesorului respectiv (indicand astfel și dimensiunea spațiului de adresare).

Conținutul acestui registru este afectat de instrucțiunea executată anterior (pe parcursul delurării programului), de apariția unor evenimente care necesită tratare (întreruperi), respectiv de mecanismul de reset în situația de inițializare a procesorului (încărcarea inițială a registrului PC).

Astfel la unele procesoare registrul $PC \leftarrow 0H$. În felul acesta prima instrucțiune pe care procesorul o va executa după o inițializare urmează să fie extrasă din memorie de la locația cu adresa 0. Într-o astfel de situație, sistemul trebuie în mod obligatoriu să aibă începând cu adresa 0 de memorie un circuit ROM. Un astfel de procesor cu această comportare este Z80.

La procesoarele 8086 (x86), în situația de reset $PC \leftarrow FFFFE_h$, conținutul PC este de 20 biți indicând un spațiu de adresare de 1M. Procesorul va executa prima instrucțiune după un reset prin preluarea conținutului de la penultima locație de memorie din spațiul de $1M=2^{20}$.

Atunci când este abordat un nou procesor trebuie studiat comportamentul PC la inițializare.

Modul de actualizare al conținutului PC-ului depinde de execuția instrucțiunii anterioare atunci când se parcurge un program de lucru. Astfel de cele mai multe ori instrucțiunile executate sunt înlanțuite una după cealaltă, fără ca acest fapt să implice salturi în memorie peste un număr mare de locații.

Instrucțiunile sunt reprezentate în memoria calculatorului pe un anumit număr de octeți funcție de tipul procesorului. În această situație conținutul PC-ului se actualizează prin adunarea numărului de octeți prin care instrucțiunea este reprezentată în memorie la conținutul anterior.

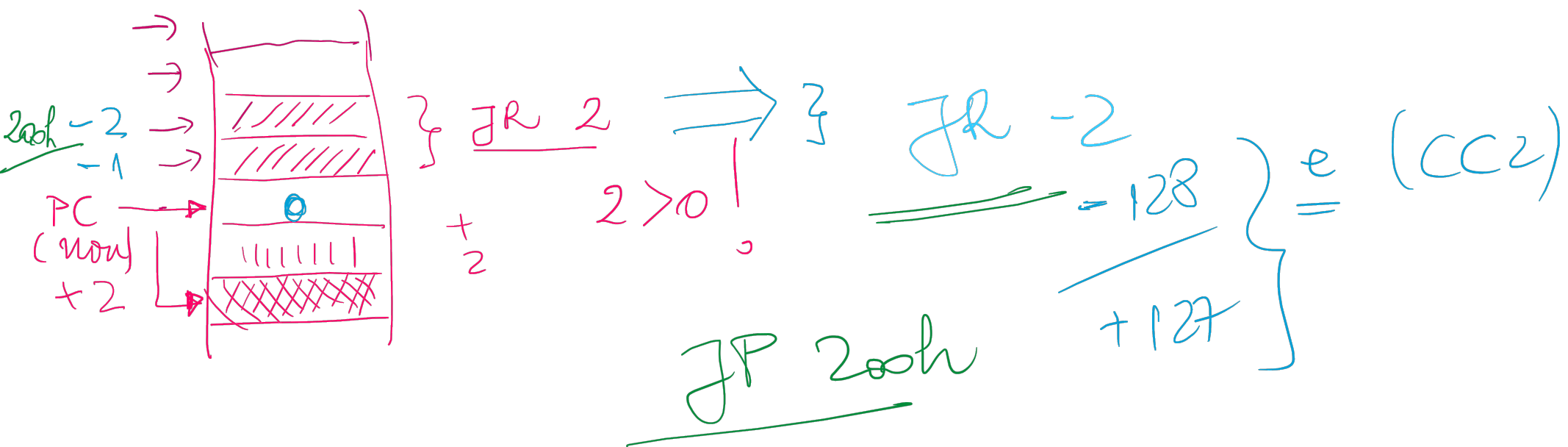
O a doua situație de actualizare al PC-ului se întâmplă atunci când instrucțiunile executate realizează salturi la parcurgerea memoriei. Sunt mai multe tipuri de instrucțiuni care pot provoca salturi la parcurgerea memoriei instrucțiunii din această categorie poartă denumirea de jump/branch (mnemonici JP/JMP și BR). Aceste tipuri de instrucțiuni au atașată pe lângă mnemonică și adresa de salt cu care PC-ul trebuie să se încarce. O astfel de încărcare a PC-ului cu adresă de salt exprimată în mod direct se numește **salt absolut**.

O altă categorie de instrucțiuni sunt cele de tip salt condiționat: JP condiție ce trebuie testată.

Toate procesoarele prezintă și instrucțiuni de salt relativ. Recunoaștem aceste instrucțiuni prin litera R existent în mnemonică instrucțiunii. Specific instrucțiunilor de **salt relativ** este faptul că în codul instrucțiunilor apare un număr binar (notat e denumit deplasament) și care este un număr reprezentat în CC2.

Acest număr arată distanța la care este plasată în memorie următoarea instrucțiune de executat de către procesor în cu instrucțiunea ce urmează după instrucțiunea curentă. Deoarece " e " este un număr în CC2, noua instrucțiune poate fi plasată către adresa de început, adică înapoi (când $e < 0$) sau către înainte ($e > 0$).

Folosind instrucțiuni cu adresare relativă un program poate fi relocat oriunde în memoria unui sistem de calcul și beneficiază de proprietatea de relocabilitate.



1000h JR -2 JP 200h - relocate to

În sinteza, afectarea conținutului registrului PC poate fi făcută în următoarele moduri:

- ✓ La inițializarea procesorului (reset) când PC-ul se încarcă cu adresa de memorie de la care este scris programul de lucru al acestuia; respectiva adresă diferă de la procesor la procesor, iar zona corespunzătoare de memorie trebuie să fie de tip ROM;
- ✓ La parcurgerea programelor de lucru, după execuția instrucțiunilor ce nu presupun efectuarea de salturi, actualizarea conținutului registrului PC se face plecând de la conținutul anterior la care se adaugă numărul de octeți pe care se reprezintă în memorie instrucțiunea executată; la procesoarele RISC acest număr rămâne întotdeauna constantă, la procesoarele CISC numărul depinde de tipul instrucțiunilor executate;
- ✓ În situația execuției instrucțiunilor ce presupun efectuarea de salturi, conținutul PC este încărcat cu adresa la care trebuie să se sară în programul de lucru; această adresă se încarcă direct în PC pentru situațiile de salt absolut, respectiv este făcută o sumă între PC-ul instrucțiunii care urmează după instrucțiunea curentă și distanța de salt (deplasament) pentru instrucțiunile de salt relativ;
- ✓ Pentru situația operării cu subrutine, conținutul PC-ului se salvează în memoria de tip stivă la apelul acesteia, iar adresei subrutinei se încarcă în PC. La sfârșitul execuției subrutinei conținutul anterior salvat în stivă se reîncarcă în PC continuându-se programul curent de lucru.

END 4 nov 2025